Live Checklist

Monday, December 8, 2025 4:39 PM

Short term firmware improvements

- [x] **Control everything in GIT**

- [ ] Motor ramp control
- [ ] Direction input from NFC (blink when direction is incorrect
- [ ] Debounce for the initialize button
- [ ] Update all variables being used on the main screen
- [ ] Second display should actively display data
- [ ] Write NFC settings mode
- [ ] Ambient light sensor implementation
- [ ] Temperature sensor implementation

Current goals/objectives

Firmware

- [ ] Integrate an I2C PN532 NFC library
  - [x] Read a card
  - [x] Integrate into zoetrope loop
    - [x] Write data onto a card
      - [ ] Dedicated mode accessible without re-programming
    - [x] Read zoetrope data/settings on the card
      - [x] Frames/rotation
      - [x] Strobe mode
      - [ ] Direction
      - [ ] Speed
      - [ ]

- [ ] Motor Control
  - [ ] Write PID loop for motor using output frequency as the target
    - [ ] Simple p loop at least
  - [x] Achieve 1000 rpm

~~Replace Stepper with BLDC (decided against this)~~

~~Identify BLDC motor and Driver~~

~~Spin the motor~~

~~Spin the motor both directions~~

~~Complete an init function~~

Mechanical - separated by part

Corner piece, top back

- [x] Add XLR 3 pin connector
- [x] Widen cable pathway
- [x] Add cable pathway for exit through the side tubes
- [x] Soften front edges
- [x] Widen top end of the piece to allow for more space around the heat inserts
- [x] Flip revision text (its upside down when viewed and you only see it on the top pieces)
- [x] Double cable exit
- [x] Fix the mirrored text revision

Corner piece, front right

- [x] Add support for switch PCB

Firmware Work

Known Bugs

- Strange behavior in un-initialized mode after running for a period of time
  - The blue LED will toggle on/off. The state will toggle to 0 (ready) but not actually run
  - Cant program it when it hits this state - requires reboot
  - I'm guessing it is a timer overflowing

STM32

- [x] Implement the encoder input - confirm working
- [x] Implement ESP32_1 UART communication
- [ ] Implement ESP32_2 UART communication
- [ ] Implement TMC2209 UART communication
- [x] Integrate NFC
  - [x] Read and use settings from NFC
    - [ ] Frames/rotation
    - [ ] Strobe mode
    - [ ] Direction
    - [ ] speed
- [x] ~~Drive a BLDC motor~~
  - [x] ~~To fast, not precise enough~~
- [ ] Stepper motor PID control loop
  - [x] Output target is output in frequency
  - [x] Stages (scaling)
  - [x] Achieve 1000 rpm
  - [ ] Controlled ramp
- [x] Multiple drive modes
  - [x] Normal run mode
  - [x] Read nfc mode

ESP32

- [x] Install all extensions needed for VS Code development
- [x] Go through vs code ESP32 quick start guide
- [x] Understand what I need to to configure my dev board
- [x] First successful build
- [x] First successful blinky
- [x] LVGL first successful build
- [x] LVGL first successful display anything
- [x] LVGL first basic page build
- [x] UART - receive data
- [x] UART - update widgets with STM32 UART data stream

Corner Piece, general

- [x] Widen cable pathway
- [x] Add cable pathway for exit through the side tubes
  - [x] Concentric 10mm hole
- [x] Flip revision text (see note on corner piece, top back)
- [x] Double cable exit
- [x] Fix the mirrored text revision

Stadium Light

- [x] Add screw locations to lock in place
  - [x] Consider just drilling and adding the screws until we are ready to re-wire
- [x] Reduce cable length between lights
- [ ] PCB Light Bars
  - [ ] Connectorize
  - [ ] Custom PCB - using the nice LEDs
  - [ ] Use same LED arrangement as original Saber

Platter Coupler

- [x] MORE screws (springs are flexing things)
- [x] Adding one more spring pin could allow for precise indexing
  - [x] Could also just adjust the spacing (from the center) for each pin to make it unique and re-make the platter I currently have (not a huge deal)
  - [x] Ive added a color indicator
- [ ] Opportunity for a neat color pattern or text
- [ ] Slip ring pass electrical connections to platter coupler
  - [ ] Pogo pins for passing electrical connections from coupler to platter
    - [ ] Pogos likely mounted into pcb inside coupler
    - [ ] Connectors for wire connections from slip ring to coupler

Main Stadium

- [ ] Hide zip tie mounts inside the print
- [ ] Integrate the NFC module
  - [ ] Hide cable well
- [ ] Larger cable way for the HALL sensor
- [ ] Can we attach the HALL sensor without glue?
- [ ] Integrate USB hub
- [ ] Zip tie mounts in vertical columns
- [ ] Lip at bottom of legs to better hold in cables
- [ ] Extra bar support to front panel mount, near top for flex reasons
- [ ] Integrate relay module
- [ ] Integrated WAGO mounts?
- [ ] Platter Presence sensing system

Front Panel Mount

- [ ] Increase cutout on bottom surface for better access to PCB (not visible)
- [ ] Exit hole for STM32 programming cable
- [ ] Adjust fit to main stadium
- [ ] Integrate USBC programming Port

Motor mount

- [ ] Slip ring integration

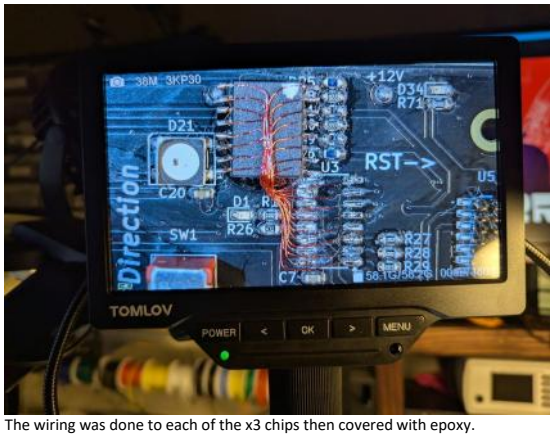
TCA9534 I2C I/O Expansion

Tuesday, December 23, 2025 2:19 PM

Connections

The TCA9534 is an I2C 8-bit I/O expansion chip. There are x3 of these chips on the V4 of the zoetrope carousel

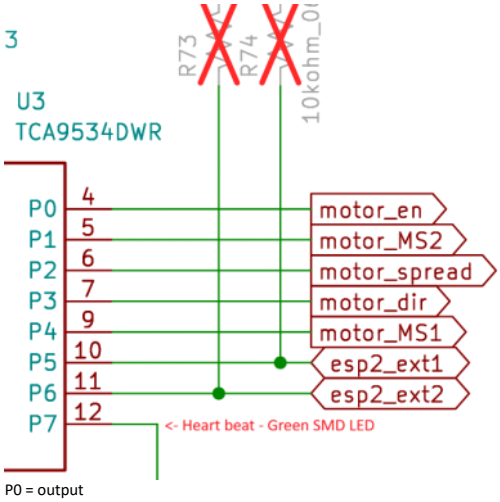
The footprint used in the design is unfortunately incorrect to the actual part. The symbol is also incorrect. The part is larger (wider) than the footprint and the SCL and SDA pins are swapped (I swapped the pins on accident in the schematic and didn't notice). These issues were accommodated for via dead-bug wiring



The wiring was done to each of the x3 chips then covered with epoxy.

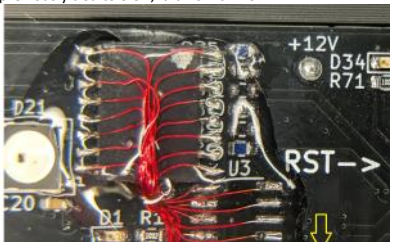
There are 3 of these I/O

U3 || TMC2209 Motor Driver control & Round ESP32 I/O connections



P0 = output
P1 = output
P2 = output
P3 = output
P4 = output
P5 = TBD (default input)
P6 = TBD (default input)
P7 = output - heartbeat LED
INT = STM32 PC5
Address = 0100000 (original schematic has duplicate addresses, this is updated address)

There was a mistake in the design and this and another IC has the same address. A trace was cut and a short added to set all address pins to 0 on U3. R28 was previously tied to 3.3V, it is now 0VDC.



I2C Driver

The TCA9534 is controlled and communicated with via 4 8 bit registers.

CONTROL REGISTER BITS		COMMAND BYTE (HEX)	REGISTER	PROTOCOL	POWER-UP DEFAULT
B1	B0				
0	0	0×00	Input Port	Read byte	XXXX XXXX
0	1	0×01	Output Port	Read/write byte	1111 1111
1	0	0×02	Polarity Inversion	Read/write byte	0000 0000
1	1	0×03	Configuration	Read/write byte	1111 1111

Register 0x00
Read only register. The bits in this register reflect the state of each I/O pin regardless if the pin is configured as input or output.

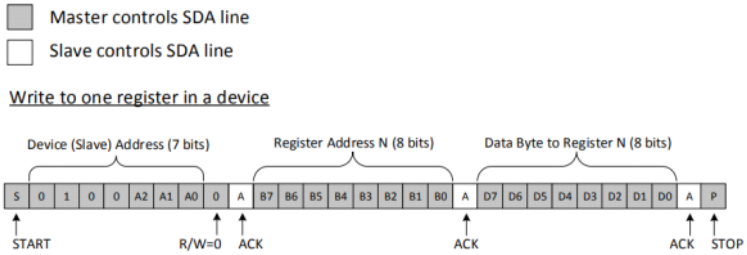
Register 0x01
Read/write register. Reads or sets the state of I/O ins already configured as outputs

Register 0x02
Read/write register. Inverts output polarity for each I/O pin

Register 0x03
Read/write register. Configuration for each I/O pin to set as input or output. 0 = output, 1 = input

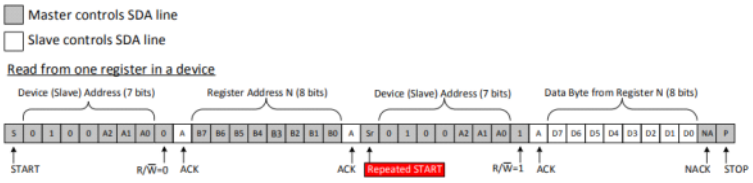
Data Transfer
I2C uses 7-bit addresses with the 8th bit being the read/write (R/W) bit. A 0 for the R/W bit indicates a write operation while a 1 indicates a read. The seven bit address is shifted left by one bit

Write Transfer



Code example:

Read Transfer

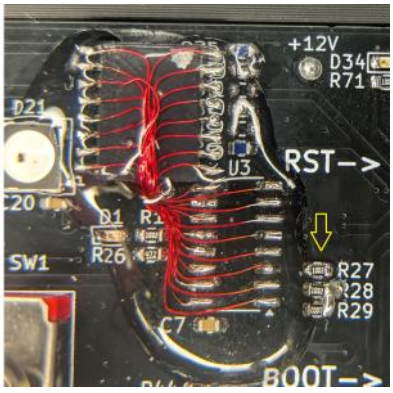


Code example:

Functions to write
- Write register
- Read register
- Read register bit (individual bit)
- Write register bit (individual bit)
- Init
  - Set configuration
  - Set polarity
  - Set default value
  - Read current input values - update relevant variables
- Zoetrope Specific functions
  - Init for each I/O chip
  - Interrupt function for each chip
    - Will check the values of each input configured pin

Initialization function
TCA9534_init(uint8_address, uint8_t confReg, uint8_t polReg, uint8_t outReg)

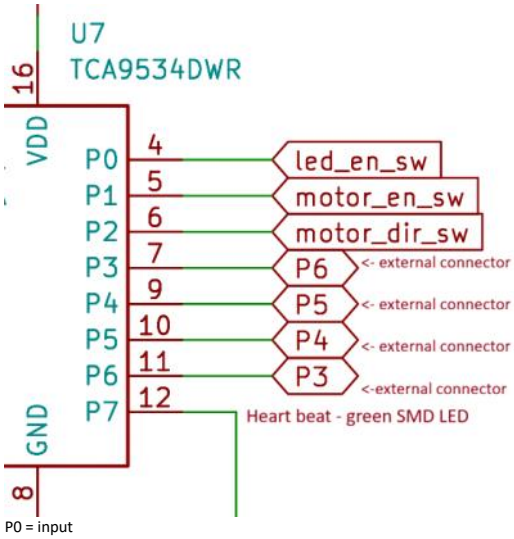
Pass in the I2C address, configuration register value, polarity inversion register value and default output register value
- Write value to configuration register
  - Configures each pin as input/output
- Write value to Polarity Inversion register
  - Configures the polarity settings for each pin
- Write value to output register
  - Set default values for each output pin



Pass in the I2C address, configuration register value, polarity inversion register value and default output register value

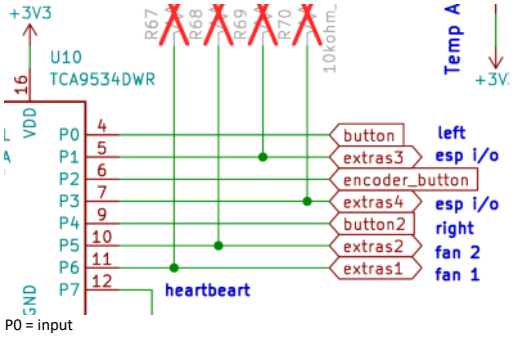
- Write value to configuration register
  - Configures each pin as input/output
- Write value to Polarity Inversion register
  - Configures the polarity settings for each pin
- Write value to output register
  - Set default values for each output pin

U7 || toggle switch connections, external future expansion I/O



P0 = input
P1 = input
P2 = input
P3 = TBD (default input)
P4 = TBD (default input)
P5 = TBD (default input)
P6 = TBD (default input)
P7 = output - heartbeat LED
INT = STM32 PC4
Address = 0100100

U10 || momentary buttons, fans and ESP32 I/O connections



P0 = input
P1 = TBD (default input)
P2 = input
P3 = TBD (default input)
P4 = input
P5 = output
P6 = output
P7 = output - heartbeat LED
INT = STM32 PC13
Address = 0100010

WS2812B NeoPixel LEDs

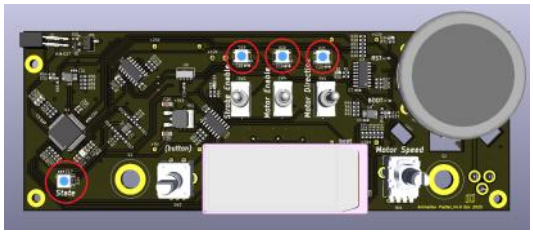
Wednesday, December 24, 2025 8:09 PM

Connections and Hardware

There are four WS2812B NeoPixel LEDs on the Zoetrope Carousel board. The neopixel line is also extended via a connector on the bottom side of the PCB.

These LEDs are full-color and individually addressable They use a single wire for communication via a PWM -like interface. The PWM signal runs at a speed of 800khz I will use PWM generated on TIM5 and DMA to send the LED data to the LEDs.

I will be following this tried and tested tutorial: https://controllerstech.com/interface-ws2812-with-stm32/
It also has a YouTube video:

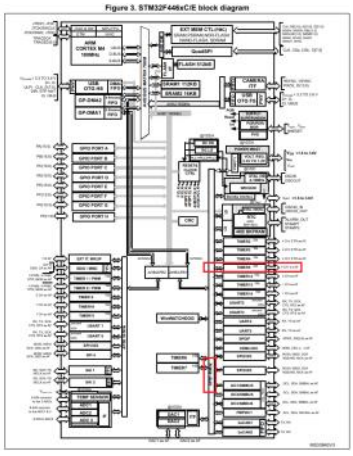


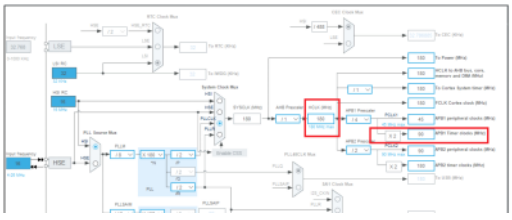
The Neopixels are connected in the order they are placed from left to right: state, strobe enable, motor enable, motor direction

The first state LED is intended to communicate status of the system. The other three are intended to communicate the status of the three switches. The line can be extended with the connector J1 on the backside of the PCB

The Neopixels are connected to pin 15, PA1, TIM5 CH2, PWM output using DMA. TIM5 is connected to the APB1 bus which runs at 90MHz with the current configuration

Figure 3. STM32F446xC/E block diagram





Ensure that TIM5 CH2 PWM output is enabled to the correct pin

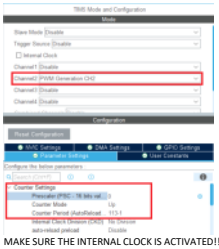
Writing the Code

Make sure the setup matches the Connections and Hardware to the left

Enable TIM5 CH2 PWM output with DMA enabled. TIM5 is connected to APB1 running at 90MHz.

Next we need to set the ARR register appropriately. The output frequency must be 800kHz.
72 MHz/X = 800 KHz || X = 112.5, round to 113

Set the Prescaler to 0 and the ARR to 113
Setting a value of 113 in the CCR register will result in 100% duty cycle



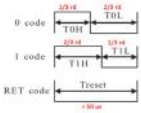
MAKE SURE THE INTERNAL CLOCK IS ACTIVATED!! (not checked in the image)

Our code will change the duty cycle of this signal to send the 0's and 1's. We need to communicate the CCR register values needed to get the desired percentage.
30% == 34
70% == 79

Next DMA must be configured appropriately so the PWM signal can be changed rapidly. Trying to update the CCR register directly will not be fast enough to send the data we need to. DMA is required

Generate the setup code

To write the code to send the data to the LEDs, we need to understand how the data looks. The WS2812 LED uses a single wire PWM -like communication interface where the PWM duty cycle's value represents either a 0 or a 1. See below



Data transfer time(TH+TL=1.25μs±600ns)

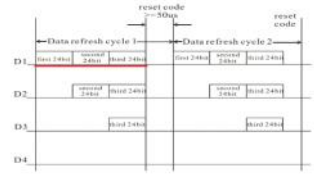
T0H	0 code ,high voltage time	0.4us	±150ns
T1H	1 code ,high voltage time	0.8us	±150ns
T0L	0 code , low voltage time	0.85us	±150ns
T1L	1 code ,low voltage time	0.45us	±150ns
RES	low voltage time	Above 50μs	

Now we know how to send each bit. Now we need to know how the data for the LEDs is structured. See below

Composition of 24bit data:

Note: Follow the order of GRB to sent data and the high bit sent at first

Data transmission method:



- You can see that each data consists of 24 bits, G R B
- And we must send the Green bits (MSB) first
- Also the data for all the LEDs should be sent together. For eg- If you have 12 LEDs, then you need to send 12×24 = 288 bits one after other
- The driver will assign the first 24 bits to the first LED, 2nd set of 24 bits to second LED, and so on
- After sending the data for all the LEDs, we must keep the signal LOW for more than 50 us, or else, the driver will consider that this data is for the NEXT LED.

The code will consist of some key variables and functions

Variables
- Number of LEDs
- 2D Array of 8-bit color data for each LED
  - LED index
  - LED R value
  - LED G value
  - LED B value
- Array of LED data to send
  - Converted from bits to CCR values
  - All color data is combined into single 16bit value

Functions
- Set RGB value for each LED, adjusts array values
- Convert RGB values to CCR values for PWM output
- Start/Stop sending LED data
- Turn All LEDs to a single color

I had a hell of a time getting this to work. Below are screenshots of my cube MX. It was also important that I declare the PWM value array (CCR values) as 32 bit. The "color" variable where the color values are converted to PWM CCR values is 32 bit and that DMA size in CubeMX is set to "Word". I also enabled auto-preload, though I'm not sure how necessary that was.

BELOW IS THE BIBLE FOR NEOPIXEL SETTINGS

THE TIMER SOURCE CLOCK IS RUNNING AT 90MHz
CCR 1 time = 56
CCR 0 time = 32

The Code - Breakdown

Much of my code is derived from the code written by Controllers Tech, the tutorial previously mentioned. For my code, I have elected to remove the brightness control. For my application I don't find it that useful and don't anticipate needing it.

Some Definitions

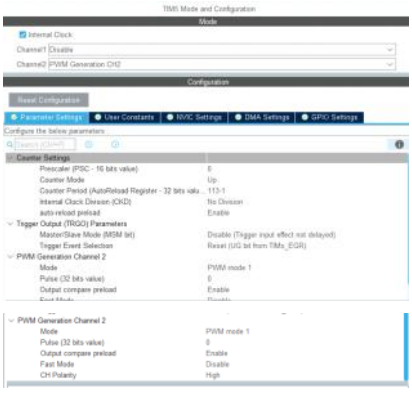
#neopixel_count = 4 //number of Neopixels
uint8_t LED_Data[neopixel_count][4]; //RGB color for each LED 8 bit per color
uint16_t pwmData[(24*MAX_LED)+50]; //contains the PWM data to send to neopixels including the end frame
uint16_t CCR_HIGH_val = 79; //CCR value for HIGH (1) transmission
uint16_t CCR_LOW_val = 34; //CCR value for LOW (0) transmission
Uint8_t datasentflag = 1; //this flag is for detecting if the DMA is done sending data

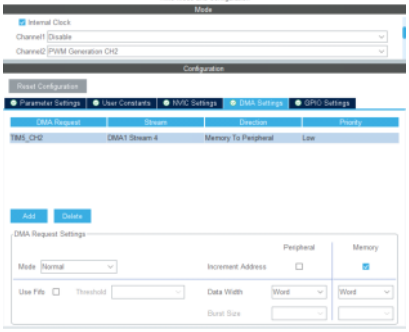
Some functions

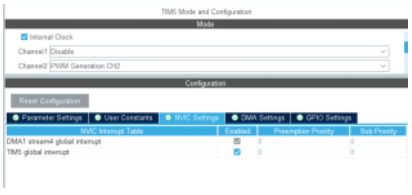
neopixel_set_LED(index, red, green, blue)
Sets the RGB values for an LED in the color matrix

Neopixel_show(void)
Converts the RGB values to PWM CCR values and outputs the data via PWM DMA

HAL_TIM_PWM_PulseFinishedCallback is used to stop the DMA transfer


TIM5 Mode and Configuration
Mode
Internal Clock
Channel1 Disable
Channel2 PWM Generation CH2
Configuration
Reset Configuration
Parameter Settings
User Constants
NVIC Settings
DMA Settings
GPIO Settings
Configure the below parameters :
Counter Settings
Prescaler (PSC - 16 bits value) 0
Counter Mode Up
Counter Period (AutoReload Register - 32 bits valu... 113-1
Internal Clock Division (CKD) No Division
auto-reload preload Enable
Trigger Output (TRGO) Parameters
Master/Slave Mode (MSM bit) Disable (Trigger input effect not delayed)
Trigger Event Selection Reset (UG bit from TIMx_EGR)
PWM Generation Channel 2
Mode PWM mode 1
Pulse (32 bits value) 0
Output compare preload Enable
PWM Generation Channel 2
Mode PWM mode 1
Pulse (32 bits value) 0
Output compare preload Enable
Fast Mode Disable
CH Polarity High


TIM5 Mode and Configuration
Mode
Internal Clock
Channel1 Disable
Channel2 PWM Generation CH2
Configuration
Reset Configuration
Parameter Settings
User Constants
NVIC Settings
DMA Settings
GPIO Settings
DMA Request
Stream
Direction
Priority
TIM5_CH2
DMA1 Stream 4
Memory To Peripheral
Low
Add
Delete
DMA Request Settings
Peripheral
Memory
Mode Normal
Increment Address
Use Fifo
Threshold
Data Width
Word
Word
Burst Size


TIM5 Mode and Configuration
Mode
Internal Clock
Channel1 Disable
Channel2 PWM Generation CH2
Configuration
Reset Configuration
Parameter Settings
User Constants
NVIC Settings
DMA Settings
GPIO Settings
NVIC Interrupt Table
Enabled
Preemption Priority
Sub Priority
DMA1 stream4 global interrupt 0 0
TIM5 global interrupt 0 0

1.91" rect. ESP32-S3 Module/Display

Tuesday, December 30, 2025 6:40 PM

This is the main display of the zoetrope carousel. It will be primarily how the user interacts with the system. It will display system variables and system menus

The zoetrope carousel can support both the touch and non-touch versions of the display. However, the zoetrope carousel is intended to use the touch version.

At a glance
Manufacture: WaveShare
Part Number: ESP32-S3-Touch-AMOLED-1.91-M
Manufacture Listing
Manufacture Wiki

Getting started - quick and dirty

Waveshare has examples that use VS code and ESP-IDF. I'm going to first attempt to upload the example project from waveshare onto the screen using vs code and ESP-IDF. If I can do that, I should be able to add or edit the project to make what I need. Better yet, I plan to actually make a clean project written nearly entirely by me

First lets
- [x] Install the extension
- [ ] Read the tutorial
- [x] Download the example
- [ ] Open in vs code
- [ ] Build the code
- [ ] Attempt upload
- [ ] Successful upload

Then we make our own basic layout
- [ ] Clear screen - make black background color standard for now
- [ ] Hello world program
- [ ] Write a few values to the screen, include labels for each of the values
- [ ] Pull values from an array

Start investigating communication from the STM32 to the ESP32. This us UART and I've proven this works in the past

ESP32 UART Packets

Saturday, January 3, 2026 4:31 PM

I need to plan a few packet structures as I write communication code between the STM32 and ESP32 screens

There are some basic ones I'm going to leave as is (not configured) and I will add a another one for initial testing

The function I'm using is structured like this:

```
packet_send(packet_type_t type, const uint8_t *payload, uint16_t len)
```

The packet types available:

```
 MSG_SYSTEM_DATA = 0x01,
MSG_TOUCH_EVENT = 0x02,
MSG_HEARTBEAT   = 0x03,
MSG_DEBUG_TEXT  = 0x04,
MSG_TESTING_DATA = 0x10
```

I'v added MSG_TESTING_DATA for initial testing as I send variables from one system to another. When I send this command, it will be the bare minimum

At a minimum, to control the zoetrope system we need the following variables sent to the ESP32 screens

- System Initialization Status || uint8_t
- System state status || uint8_t
- Platter rotation period || uint32_t
    - RPM, Hz, frameTime, ect are calculated from platter rotation and displayed
- Platter frame count || uint8_t
- Motor speed setting || uint16_t setpoint of the potentiometer
- Strobe enabled || uint8_t
- Motor enabled || uint8_t
- Motor direction || uint8_t

That is 1, 1, 4, 1, 2, 1, 1, 1 = 12 bytes

ESP32 ESP-IDF Project setup

Monday, January 5, 2026 2:34 PM

Copy base project to the new destination, rename the folder

The current main c project is called
example_qspi_with_ram

Do a full project clean, then attempt to build.. There are a few changes that will need to happen if they havent already

Include "ui.h" in the main c file
Update the cmakelists.txt file to include the ui files

main > M CMakeLists.txt

```
idf_component_register(
    SRCS
        "example_qspi_with_ram.c"
        "ui/ui.c"
        "ui/ui_events.c"
        "ui/ui_helpers.c"
        "ui/ui_Screen1.c"
    INCLUDE_DIRS
        "ui"
        "."
)
```

The above files are generated by the squareline studio project

The UI likely uses more than just the default font size. To prevent the compiler from breaking, the font size needs to be enabled

Inside of components->lvgl->src->lv_conf_internal.h the font can be enabled

```
project_elf_src_esp32s3.c
x509_crt_bundle.S
components
  adc_bsp
  esp_lcd_sh8601
  esp_touch
  esp_wifi_bsp
  espressif__cmake_utilities
  i2c_bsp
  lvgl
    demos
    docs
    env_support
    examples
    scripts
    src
      core
      draw
      extra
      font
      hal
      misc
      widgets
      lv_api_map.h
      lv_conf_internal.h
      lv_conf_kconfig.h
```

```
1068  #ifndef LV_FONT_MONTSERRAT_28
1069      #ifdef CONFIG_LV_FONT_MONTSERRAT_28
1070          #define LV_FONT_MONTSERRAT_28 CONFIG_LV_FONT_MONTSERRAT_28
1071      #else
1072          #define LV_FONT_MONTSERRAT_28 0
1073      #endif
1074  #endif
1075  #ifndef LV_FONT_MONTSERRAT_30
1076      #ifdef CONFIG_LV_FONT_MONTSERRAT_30
1077          #define LV_FONT_MONTSERRAT_30 CONFIG_LV_FONT_MONTSERRAT_30
1078      #else
1079          #define LV_FONT_MONTSERRAT_30 1
1080      #endif
1081  #endif
1082  #ifndef LV_FONT_MONTSERRAT_32
1083      #ifdef CONFIG_LV_FONT_MONTSERRAT_32
1084          #define LV_FONT_MONTSERRAT_32 CONFIG_LV_FONT_MONTSERRAT_32
1085      #else
1086          #define LV_FONT_MONTSERRAT_32 0
1087      #endif
1088  #endif
1089  #ifndef LV_FONT_MONTSERRAT_34
1090      #ifdef CONFIG_LV_FONT_MONTSERRAT_34
1091          #define LV_FONT_MONTSERRAT_34 CONFIG_LV_FONT_MONTSERRAT_34
```

```
PROBLEMS 1   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   MEMORY   XRTOS   ESP-IDF
ui_Screen1.c main\ui 1
'lv_font_montserrat_30' undeclared (first use in this function); did you mean 'lv_font_montserrat_16
```

//call the main ui task in the main c file

```
main > example_qspi_with_ram.c
318  #endif
322      xTaskCreate(example_lvgl_port_task
323
324      ESP_LOGI(TAG, "Display LVGL demos");
325      // Lock the mutex due to the LVGL APIs are not thread-safe
326      if (example_lvgl_lock(-1))
327      {
328          //setup_ui(&guider_ui);
329          //events_init(&guider_ui);
330          //user_bsp_Init(&guider_ui);
331          //lv_demo_widgets();      /* A widgets example */
332          //lv_demo_music();        /* A modern, smartphone-like music player d
333          //lv_demo_stress();       /* A stress test for LVGL. */
334          //lv_demo_benchmark();    /* A demo to measure the performance of LVG
335
336          ui_init();          // ← SquareLine entry point
337
338
339          // Release the mutex
340          example_lvgl_unlock();
341      }
342  }
343  void esp_draw_bitmap(uint16_t clorck,esp_lcd_panel_handle_t panel_handle)
344  {
345      uint16_t *buf = (uint16_t *)malloc( EXAMPLE_LCD_H_RES * EXAMPLE_LCD_V_RES
```

CONFIG_LV_FONT_DEFAU Aa ab .* No results

Verify build works

Next, add the heartbeat function. This function just pulses an LED connected to one of the ESP32 pins

For the zoetrope, each ESP-32 has a different pin connection. That will need to match the correct pin in the code we include next

Bring heartbeat.c and heartbeat.h into the project. To keep things simple, I just put them in the main folder for now

Include heartbeat.h in the main c file and Update the cmakelists.txt file to include "heartbeat.c"
//update includes

```
#include "lvgl.h"
#include "lv_demos.h"
#include "esp_lcd_sh8601.h"
#include "touch_bsp.h"
#include "i2c_bsp.h"
#include "gui_guider.h"
#include "events_init.h"
#include "user_bsp.h"
lv_ui guider_ui;

#include "ui.h" //lvgl squareline generated ui
#include "heartbeat.h" //heartbeat led code

static const char *TAG = "example";
```

Update the cmake lists

```
idf_component_register(
    SRCS
        "example_qspi_with_ram.c"
        "ui/ui.c"
        "ui/ui_events.c"
        "ui/ui_helpers.c"
        "ui/ui_Screen1.c"
        "heartbeat.c"
    INCLUDE_DIRS
        "ui"
        "."
)
```

Add the heartbeat task to the main c task list (whatever that is called)

```
    // Lock the mutex due to the LVGL APIs are not thread
    if (example_lvgl_lock(-1))
    {
        //setup_ui(&guider_ui);
        //events_init(&guider_ui);
        //user_bsp_Init(&guider_ui);
        //lv_demo_widgets();      /* A widgets example */
        //lv_demo_music();        /* A modern, smartphone
        //lv_demo_stress();       /* A stress test for LV
        //lv_demo_benchmark();    /* A demo to measure th

        heartbeat_init(); //heartbeat led code
        heartbeat_start();

        ui_init();          // ← SquareLine entry point


        // Release the mutex
        example_lvgl_unlock();
    }
}
void esp_draw_bitmap(uint16_t clorck esp_lcd_panel_handle
```

Test that the build works

Next the uart communication can be added to the project. There are three files that do the UART configuration and communication. For simplicity, I added them into the main filder as well
Uart_protocol.h
Uart_protocol.c
Packet.c

In uart_protocol.h, make sure these parameters match the ESP32 being programmed

```
#include "driver/uart.h"

/* UART config */
#define UART_PORT       UART_NUM_1
#define UART_TX_PIN     43
#define UART_RX_PIN     44
#define UART_BUF_SIZE   256

#define UART_SOF_BYTE         0xAA
#define UART_MSG_TYPE_TEST    0x10
#define UART_MAX_PAYLOAD_LEN 64
```

Add uart_protocol.h in the includes section of the main c file

Update the cmakelists file to include uart_protocol.c

Add the uart task to the main c file

```
{
    //setup_ui(&guider_ui);
    //events_init(&guider_ui);
    //user_bsp_Init(&guider_ui);
    //lv_demo_widgets();      /* A widgets example */
    //lv_demo_music();        /* A modern, smartphone-like
    //lv_demo_stress();       /* A stress test for LVGL. *
    //lv_demo_benchmark();    /* A demo to measure the per

    heartbeat_init(); //heartbeat led code
    heartbeat_start();

    uart_rx_init(); //uart communication init

    ui_init();         // ← SquareLine entry point


    // Release the mutex
    example_lvgl_unlock();
}

oid esp_draw_bitmap(uint16_t clorck esp_lcd_panel_handle_t pa
```

Verify the project will build

Next, include the code that updates the LVGL UI values with the UART values. This uses two files
Platter_ui.h
Platter_ui.c

I copy these into the main folder for simplicity

Update the includes to have platter_ui.h

```
#include "events_init.h"
#include "user_bsp.h"
lv_ui guider_ui;

#include "ui.h" //lvgl squareline generated ui
#include "heartbeat.h" //heartbeat led code
#include "uart_protocol.h" //uart rx data structure
#include "platter_ui.h" //platter ui update functions

static const char *TAG = "example";
```

Update cmakelists.txt file to include "platter_ui.c"

```
idf_component_register(
    SRCS
        "example_qspi_with_ram.c"
        "ui/ui.c"
        "ui/ui_events.c"
        "ui/ui_helpers.c"
        "ui/ui_Screen1.c"
        "heartbeat.c"
        "uart_protocol.c"
        "platter_ui.c"
    INCLUDE_DIRS
        "ui"
        "."
)

set_source_files_properties(
    ${LV_DEMOS_SOURCES}
    PROPERTIES COMPILE_OPTIONS
    -DLV_LVGL_H_INCLUDE_SIMPLE)
```

Update the main tasks section to include platter ui updates

```
    if (example_lvgl_lock(-1))
    {
        //setup_ui(&guider_ui);
        //events_init(&guider_ui);
        //user_bsp_Init(&guider_ui);
        //lv_demo_widgets();      /* A widgets example */
        //lv_demo_music();        /* A modern, smartphone-li
        //lv_demo_stress();       /* A stress test for LVGL.
        //lv_demo_benchmark();    /* A demo to measure the p

        heartbeat_init(); //heartbeat led code
        heartbeat_start();

        uart_rx_init(); //uart communication init
        platter_ui_init(); //platter ui update functions

        ui_init();         // ← SquareLine entry point


        // Release the mutex
        example_lvgl_unlock();
    }
}
```

Build project - should work

Live programming notes

Friday, January 9, 2026 12:05 PM

The sys Init variable being sent to the screen seemed random. I thought it was working at one point, but I have not investigated further. I just noticed that the TIM1->CNT which gets assigned to the encoder value seems to be changing the sys init variable. The correlation to change is direct, but the value at which the change occurs is not entirely consistent with the value of change to the encoder value

Strobe Control Algorithm

Friday, January 9, 2026 6:14 PM

V4 of the zoetrope carousel is going to have a new strobe algorithm to tighten up the performance of the sytem.

The V3 algorithm was good, but had a few pitfalls. The most important one we need to address/ovoid is every once in a while the strobe would get stuck on for one rotation. This is super jarring and disruptive and not acceptable for this version.

The above issue was not fully figured out. My best guess currently was due to large remainders in the math determining the strobe timing. For example, to calculate the strobe timing, the timing period would be divided by the number of frames in a rotation. At certain speeds (period) the issue would occur. I'm assuming this was when there was a large remainder in this frame time calculation.

For version 4, we will perform this same calculation, but take the remainder into account. We will take the remainder, and average the error across all frames in one rotation.

The strobe for V4 will be controlled by a PWM timer. Settings for this PWM will be calculated based on the speed of the platter, the number of frames in a rotation, and the desired on time for each frame.

For PWM output, the frequency is controlled from the PSC (prescaler) and ARR (Auto Reload) registers. The duty cycle is controlled by the CCR (capture compare) register.

- **PWM frequency** tells how many times the signal completes a full cycle in one second. It depends on three parameters: timer clock, prescaler (PSC), and ARR.
  Formula: $f_{PWM} = \frac{f_{timer}}{(PSC+1)(ARR+1)}$
- **Duty cycle** defines the percentage of ON-time in each cycle. It depends on the compare register (CCR) value.
  Formula: $D(\%) = \frac{CCR}{ARR+1} \times 100$

Before the frequency of the PWM output can be set, we need to calculate the needed frequency. This is going to be a factor of how fast the platter is spinning and how many frames exist in one platter rotation

Needed frequency = 1 / ((platter rotation period) / (frame count))
This is one over the period of the signal

The duty cycle will be a bit trickier to implement depending on how the math works out. The UI says we can adjust in microseconds, but we will only have control over the period in the resolution that the set ARR value can permit.. The CCR value is relative the value of the ARR register. We need to keep the ARR value as high as possible when setting the frequency in order to keep enough control over the ON period. The ON period is a very short amount of time.

We can have our adjustment as tight as we can make it, and just output what the value ends up being in microseconds for the time being. We will directly control the CCR when setting ON time, but report out the resulting calculated microsecond ON time from that CCR change

We also want to implement how often we update the motor speed. The current code uses a direct raw value from the ADC connected to the potentiometer. With the nature of an ADC, it is not always static. I think I would only want to update the motor speed, if the nob changes 0.5% or more.

This strobe pattern needs the ability to index. To accomplish this, we need to store all the pulse values for one complete rotation and send that to the PWM timer via DMA. This will limit us for how fast we can update the algorithm real-time, but is a limitation that already exists today.

Steps:
Take speed reading - hall reading (need to configure as interrupt)
- -overwrite PWM register with values from shadow register
- -stop DMA PWM transfer
- -Start DMA PWM transfer from beginning of array
- -re-calculate into shadow array with new speed data

NEW Variables
- -uint16_t shadowPWMArray[255]
- -uint16_t PWMArray[255]
- -strobe_CCR
- -strobe_ARR
- -strobe_PSC

Existing used variables
- -platter rotation time
- -on time setting
- -slice count

I think I want to put this into its own files at some point

GOT THIS WORKING - IT WORKS VERY WELL

NFC - PN532

Friday, January 16, 2026 6:39 PM

I have had the brilliant idea of using NFC with the zoetrope. Previously, I was going to place a series of magnets on the Platter to read the platter's ID. I have now decided to use NFC stickers placed on the outside (for existing platters), and placed INSIDE of the print on new platters. The ID and settings can then be written onto the NFC stickers and read in by the system. This allows the Zoetrope Carousel to be fully autonomous, no matter how unique the platter. Not only this, but I already included two I2C connections on the back of the PCB so we are already ready to rock and roll

Benefits
Automated - easier to use and less to remember
Huge cool factor
Excuse to learn NFC

Downsides
Havent found a HAL library for drop in that works (yet)
Increased complexity

Hardware:
- Reader: PN532 in I2C mode
- Tags: NTAG 215 - 504 Bytes
  - Stickers: https://www.amazon.com/dp/B0F24L4GDM?ref=ppx_yo2ov_dt_b_fed_asin_title
  - Coins: https://www.amazon.com/dp/B0CHVWTRGC?ref=ppx_yo2ov_dt_b_fed_asin_title&th=1

Software:
- Library reference/starting point: https://github.com/soonuse/pn532-lib

Theory for NFC in my system
- Detect when the platter has been changed
- Spin 1 full rotation at a slow speed, while scanning and register the NFC tag
  - This way the code can slow everything down temporarily
  - Read data from NFC tag and apply settings
- If no tag is detected, enter default manual mode
- If tag is detected, read it and apply the contained settings- spin into automatic mode

NTAG 215 Data transfer theory
- My tags have 504 Bytes of available storage data
- NTAG 215 tags store data in "pages" each page is 4 byes
  - Data must be read and written in page lengths (4 byte chunks)
  - If less than 4 bytes must be read/written, the entire page must be manipulated
- User for my tags begin at page 4
- See the next section for the data structure planning, each version will have its own section

Code Development

Ideally I will start from a HAL library. Otherwise, I will take a working Arduino Library and adapt it for HAL

I have confirmed this video and library works with Arduino:
https://how2electronics.com/interfacing-pn532-nfc-rfid-module-with-arduino/
-it is an arduino library, I would need to adapt it to use HAL I2C functions, everything else should just "work"

The above library is not only used by Arduino, but it is also written in C++. This makes some additional challenges. I ether have to update my main project to use C++ or convert the library to C.

I had ChatGPT convert the library to C. It changed more than just that. I was able to get it to compile and upload, but never really work.

I did connect the oscilloscope to the clock line at the PN532. I compared the signal at that point to another point near the microcontroller itself. There was noise for sure on the line, but it really seemed, like the noise shouldn't be too bad to mess up the system. With that, I have assumed that ChatGPT did not give me what I needed.

This library Has an STM32 Option. At first glance in the readMe, you would guess it only has SPI for STM32, but it actually has I2C. Lets give this one a shot. It appears to be a Waveshare library
https://github.com/soonuse/pn532-lib/tree/master

This library should be more plug n' play. Just need to learn how to use it

I got the code to run successfully after commenting out all the lines for SPI, UART and the IRQ and RST GPIO calls.

I uploaded the code and integrated it simply into the zoetrope loop, but it is REALLY slow. It either needs optimization, use of the IRQ pin, or I have to know exactly when the card is in front of the reader (location and indexing based)

Zoetrope Carousel Platter NFC Tag Data Transfer Structure

V1.0

Each row is reserved for a page, each cell is a byte
Far left column is page number the other four are bytes within the page
My tags have 504 bytes which is 126 pages of data

Page Index	Page Description	Byte 3	Byte 2	Byte 1	Byte 0	Comment
1						We get Platter ID, but just using the NFC ID
2						
3						
4						
5						Library example starts writing data at page 6, replicating that to be safe here
6 (data structure revision)	Data structure revision	0x00	0x10	0x01	0x00	Revision of the data structure, for zoetrope to know how to handle the data from the NFC sticker
7 (platter)	Platter revision	0x**	0x**	0x**	0x**	Revision of the platter data and physical state
8 (speed range)	Top and bottom speed	0x**	0x**	0x**	0x**	Speed range (user input available rang)
9 (ideal range)	Pot Top center speed	0x00	0x00	0x**	0x**	
10 (frame_1)	Frame Settings	0x**	0x**	0x**	0x**	Frame settings
11 (frame_2)	Frame Settings	0x00	0x00	0x00	0x**	Frame settings
12 (PID_P)	PID P value	0x**	0x**	0x**	0x**	PID Value for platter spin up
13 (PID_I)	PID I Value	0x**	0x**	0x**	0x**	PID Value for platter spin up
14 (PID_D)	PID D Value	0x**	0x**	0x**	0x**	PID Value for platter spin up
15 (brightness level)	Reserved, empty page	0x00	0x00	0x00	0x**	Empty but sent with packet
16 (Reserved)	Reserved, empty page	0x00	0x00	0x00	0x00	Empty but sent with packet
17 (N/A)	--	--	--	--	--	NFC data V1.0 uses pages 1-16, rest is ignored
18 (N/A)	--	--	--	--	--	Not manipulated yet
... (N/A)	--	--	--	--	--	Not manipulated yet
126 (TBD)	--	--	--	--	--	Last page available on sticker

(6) Data Structure Revision
- Byte 0 - data structure - minor revision number
- Byte 1 - data structure - major revision number
- Byte 2 - data structure - page count in revision (how much data to read on the card for full data read)
- Byte 3 - TBD (reserved)

(7) Platter
- Byte 0 - platter - settings minor revision number
- Byte 1 - platter - settings major revision number
- Byte 2 - platter - physical minor revision number
- Byte 3 - platter - physical major revision number

(8) Speed Range
- Byte 0 - Low range LSB (units?)
- Byte 1 - Low range MSB (units?)
- Byte 2 - high range LSB (units?)
- Byte 3 - high range MSB (units?)

(9) Ideal Speed (top center of potentiometer)
- Byte 0 - ideal range LSB (units?)
- Byte 1 - ideal range MSB (units?)
- Byte 2 - TBD
- Byte 3 - TBD

(10) Frame_1
- Byte 0 - Frame count
- Byte 1 - Strobe mode
- Byte 2 - on time LSB
- Byte 3 - on time MSB

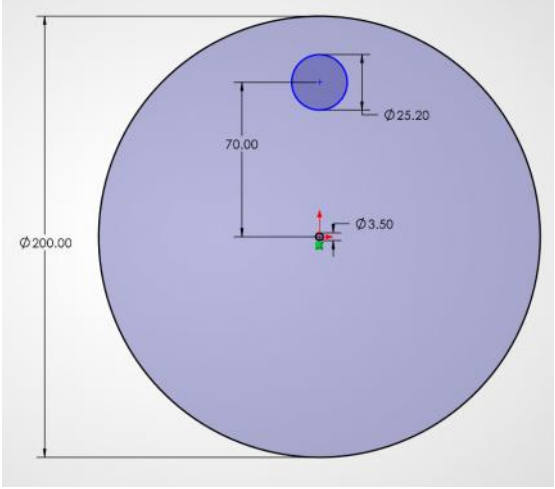
(11) Frame_2
- Byte 0 - Direction
- Byte 1 -
- Byte 2 -
- Byte 3 -

(12) PID_P
- Byte 0 -
- Byte 1 -
- Byte 2 -
- Byte 3 -

(13) PID_I
- Byte 0 -
- Byte 1 -
- Byte 2 -
- Byte 3 -

(14) PID_D

Platter Mechanical Specifications (NFC)



UPDATE THIS IMAGE WHEN WE FIGURE OUT WHERE THE SPRING PIN INDEX IS - ALSO INCLUDE MAGNET SILLOUETTE
NFC void is 1.2mm deep. .2mm of tolerance around the coin. Sticker should also fit the same void
Ideal platter total thickness is between 2 and 3mm

Using NTAG2 NFC Tags. Two form factors - Stickers and coins

Stickers are used for already existing platters - allows for reverse compatible
Stickers: https://www.amazon.com/dp/B0F24L4GDM?ref_=ppx_hzsearch_conn_dt_b_fed_asin_title_3
Coins: https://www.amazon.com/dp/B0CHVWTRGC?ref_=ppx_hzsearch_conn_dt_b_fed_asin_title_1&th=1

Byte 0 -
Byte 1 -
Byte 2 -
Byte 3 -

(15) brightness (some platters look better with lower brightness)
Byte 0 - brightness
Byte 1 -
Byte 2 -
Byte 3 -

(16) Reserved (sent, but always 0 for now)
Byte 0 -
Byte 1 -
Byte 2 -
Byte 3 -

Motor Drive Control Loop

Sunday, January 18, 2026 7:58 PM

Stepper Motor Control Loop implementation
The current code directly maps the value read from the potentiometer to the CCR register of the timer that outputs to the TMC2209 stepper driver step pin. This works, but has a few disadvantages
- The speed setting is not linear. This is just due to how the registers in the timers behave
- The increases at the top end of the potentiometer become very large. The large increases/decreases make it easy to stall the motor

Instead of directly mapping the value, we want to map the potentiometer to a frequency setting, then map that to proper register values. We can then use a control loop, such as PID, to control how quickly we change the output frequency to prevent stalling the motor.

Steps to accomplish this:
- Figure out the math to map potentiometer values to proper register values in the timer to achieve the desired output frequency
- Write PID functions and add PID variables and tracking
- Implement motor drive mode switching based on the system motorMode variable

Lets add some checkboxes
- [x] Add PID tracking variables
- [x] Add PID values variables
- [ ] Add/define motor drive modes and assign variable values
- [ ] Add system switching of the motor mode based on the system variable
- [ ] Figure out the math to set desired output frequency via TIM registers
- [ ] Write PID system functions
- [ ] Write a motor mode that uses PID to control the system speed to control the motor speed with a desired frequency setting
- [ ] We will need a system drive setting that will allow for automatic motor mode switching

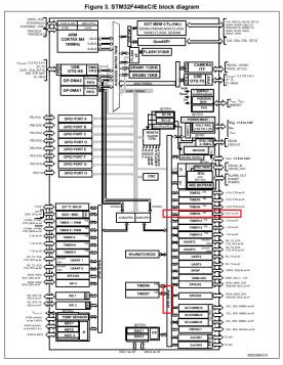
Some handy references
- My Google drive, I wrote an article that covers all I should need, just for this purpose

Figuring out the math for the new step algorithm. The new algorithm will use PID and the input and output units will be RPM

Goal - dynamically set timer registers to achieve set RPM value

Information gathering
- Pin: PA5; 21; TIM8 CH1N; PWM output
- TIM8 frequency: APB2 90MHz
- Target Duty Cycle: 50%
- CCR Value: ARR / 2
- Live manipulate PSC and ARR to achieve RPM
- Stepper motor: 1.8 degree
- Steps revolution: dependent on motor settings
  - Full step
    - 200 steps/rev
  - Half step
    - 400 step/rev
  - Quarter step
    - 800 step/rev
  - Eighth step
    - 1600 step/rev
- Hz: (steps/revolution)

Figure 3. STM32F446xC/E block diagram



Chat GPT broke down the process pretty well, I'll paste the screenshots below of the overall process I'm planning to follow. I'm sure I'll make a few small adjustments as we go

Step 1: Convert RPM → STEP frequency

We generate **one step pulse per microstep**.

```ini
steps_per_rev = 200 × 8 = 1600
```

Desired RPM → step frequency (Hz):

```ini
f_step = (RPM × steps_per_rev) / 60
```

Example:
- 1000 RPM → 26,666.67 Hz

Step 2: Timer frequency equation (STM32)

For edge-aligned PWM:

```ini
f_pwm = TIM_CLK / ((PSC + 1) × (ARR + 1))
```

We want:

```ini
f_pwm = f_step
```

So:

```
(PSC + 1) × (ARR + 1) = TIM_CLK / f_step
```

Step 3: Strategy for choosing PSC & ARR

For **PID control**, you want:
- **PSC as small as possible** (better resolution)
- **ARR reasonably large** (smooth frequency changes)

Recommended strategy
- Fix `PSC = 0` initially
- Compute `ARR`
- If `ARR` is too large (> 65535), increase `PSC`

This gives:
- Maximum timing resolution
- Clean PID behavior

Step 4: Code (core calculation)

Constants

```c
#define TIM8_CLK_HZ        90000000UL
#define FULL_STEPS_PER_REV 200
#define MICROSTEPS         8
#define STEPS_PER_REV      (FULL_STEPS_PER_REV * MICROSTEPS)
#define TIM_MAX_ARR        65535
```

Data structure (optional but nice)

```c
typedef struct {
    uint16_t psc;
    uint16_t arr;
    uint16_t ccr;
    float    step_freq;
} StepTimerConfig;
```

```
#include <math.h>

void Compute_TIM8_FromRPM(float rpm, StepTimerConfig *cfg)
{
  if (rpm <= 0.0f) {
      cfg->psc = 0;
      cfg->arr = 0;
      cfg->ccr = 0;
      cfg->step_freq = 0.0f;
      return;
  }
```

Step-Configuration

CFG2/MS2	CFG1/MS1	SPREAD/MS3	Steps	Interpolation	Mode
GND	GND	GND	1/8	1/256	stealthChop
GND	VIO	GND	1/32	1/256	stealthChop
VIO	GND	GND	1/64	1/256	stealthChop
VIO	VIO	GND	1/16	1/256	stealthChop
GND	GND	VIO	1/8	1/256	spreadCycle
GND	VIO	VIO	1/32	1/256	spreadCycle
VIO	GND	VIO	1/64	1/256	spreadCycle
VIO	VIO	VIO	1/16	1/256	spreadCycle

```
    // Step frequency in Hz
    float step_freq = (rpm * STEPS_PER_REV) / 60.0f;
    cfg->step_freq = step_freq;

    uint32_t tim_clk = TIM8_CLK_HZ;
    uint32_t psc = 0;
    uint32_t arr;

    // Find smallest PSC that gives ARR within range
    while (1) {
        arr = (uint32_t)((tim_clk / ((psc + 1) * step_freq)) - 1);

        if (arr <= TIM_MAX_ARR) {
            break;
        }

        psc++;
        if (psc > TIM_MAX_ARR) {
            // Cannot generate this slow frequency
            arr = TIM_MAX_ARR;
            break;
        }
    }

    cfg->psc = (uint16_t)psc;
    cfg->arr = (uint16_t)arr;
    cfg->ccr = cfg->arr / 2;  // 50% duty
}
```

I will implement the code a bit different, but the overall concept will initially be the same

V5 (for next rev)

Tuesday, January 20, 2026 6:42 PM

List of system improvements for when the main PCB undergoes an update

- The IO expansion I2C chips had a mis-wire and incorrect footprint size
- Look at other I2C lines and DMA
    - Unable to add I2C to DMA due to lines used for DMA (timers, uart, etc)
- With the NFC, HALL sensors and optical sensor, I really think it makes sense for a second PCB, mounted on top of the platter platform
    - Contains at least x2 HALL sensors
    - Contains NFC chip and PCB antenna
    - Contains the optical sensor
    - Contains the motor drive circuitry and microcontroller
        - FOC would be great
        - My OWN ESC would solve low speed issue
    - Connections
        - Direct connections to the HALL sensors, for speed
        - I2C line for NFC reading, secondary microcontroller (motor driver)
        - Direct connection for optic sensor
    - Contain ambient light sensing?
    - USB (UART) interface
        - For debugging, would be very useful
- Add another UART (USB) interface for debugging
- TVS Diode on HALL sensor lines. Check all lines that go to an external connector
- Be able to use both 12v and 24v strobe LEDs

Platter Detection Sensor

Saturday, January 24, 2026 4:14 PM

I need to determine when a new platter has been taken on and off of the zoetrope carousel. This will allow me to only check for the NFC tag when needed, and will allow me to not overwrite settings (like direction) on each motor startup.

Without presence detection and if we take direction from NFC, we will only be able to spin each disk in a single direction, even if we want to change the direction. (because NFC is read each time the motor is disabled, which is the only time the direction can be changed)

I am planning to use an infrared presence sensor to look for the platter. I have done some initial testing and results indicate that it is possible to do this with the sensor I have. The issue, is that presence detection is far less effective when looking at a black object as opposed to a white object. All of the platters are black which causes an issue.

I was planning to use a simple GPIO input to detect the presence of the platter. This would work if all the platters were white, but they are not. Due to this, we will need to add a bit more complexity to the setup - which the hardware can support.

We need to use an ADC input to read the sensor. We also need a GPIO pin to enable/disable the IR LED used for presence sensing (to detect ambient IR light levels).

We will use an ADC input available on one of the free HALL connectors
We will use a GPIO output from one of the available I2C I/O chips.

The connections will be documented in the schematics.

I have used this instructible as a reference for my testing, which was successful with few modifications:
https://www.instructables.com/Cheap-Infared-Sensors-RPR220/

I used a 220 ohm resistor for the IR LED - as was used in the example

- This could potentially be reduced to increase performance, we are using a lower voltage level

I used a 22kohm resistor for the transistor output. I am using a more-near distance than them. This could also be tuned

These connections could be soldered to a simple perf board and wired to the PCB for initial testing. A PCB could be made in the future to clean things up

Emily had a great idea -

- Instead of adding the ability to know for sure when the platter is removed, we change how the direction switch works.
    - Instead of not allowing the direction to be changed unless the motor is disabled, when the direction is changed and the motor is spinning, the motor will automatically spin down and change its direction. This would reduce the sequence by a step and make the system easier to use
        - The direction light, could continue to blink until the new direction has fully taken

Clock Platter Strobe Mode

Saturday, January 31, 2026 1:10 PM

This is brainstorming and explanation of the strobe algorithm required to run the clock platter designed jan 2026

The platter is a clock with 5 digit positions. Two for hour, two for minute, one separator. The clock is intended to be placed directly in front of the user

The platter design has the seven segment pattern repeated four times. Each segment is sandwiched between four blank spaces. Total of 105 "slices"

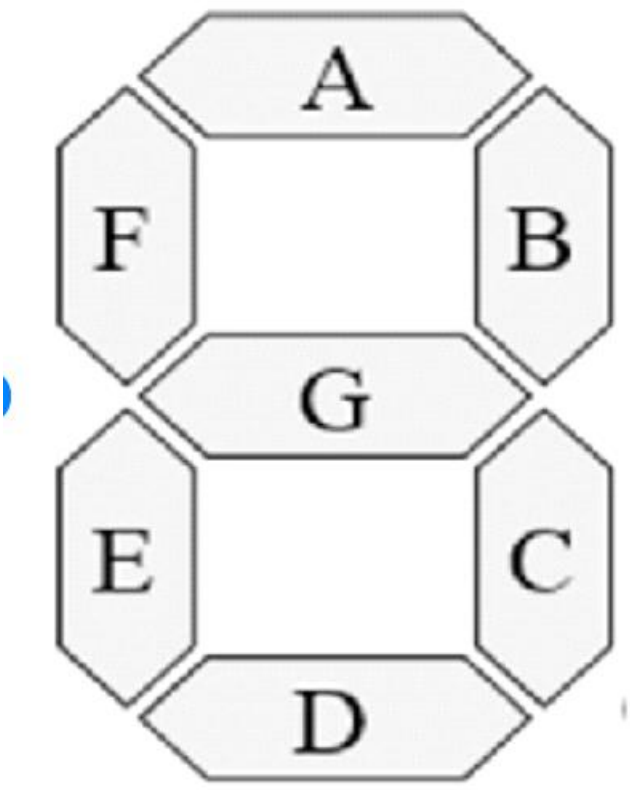
Target refresh frequency: 30hz

With the pattern repeating three times, we need to spin the platter at 10hz

The algorithm must generate a full rotation's worth of pulses on each pulse refresh. Below is the flow I'm currently thinking

The HALL effect trigger is very important. With the locking platter, it is how we know where the platter is located. We will eventually need to calculate an offset from this point so that we can place the clock display where we want it to be. To start, I will not worry about an offset, I will add that at the end

The hall sensor trigger indicates the beginning of the first seven segment pattern. The pattern repeats itself 3x, A, B, C, D, E, F, G - CLOCKWISE ROTATION



Decimal	Segments
0	ABCDEF
1	BC
2	ABDEG
3	ABCDG
4	BCFG
5	ACDFG
6	ACDEFG
7	ABC
8	ABCDEFG
9	ABCDFG

The clock display will exist in 5 distinct slice index points. To start, I am going to use index slices 0, 1, 2, 3,

4 and apply an offset later to accurately place the clock

First we need to note what index on the rotating platter each slice exists in. That is recorded below. Everything is 0 based (start at 0)

A - 0, 35, 70
B - 5, 40, 75
C - 10, 45, 80
D - 15, 50, 85
E - 20, 55, 90
F - 25, 60, 95
G - 30, 65, 100

As the platter rotates, the pulse timer will automatically fire. IF it fires is dependent on if a segment used in the clock display, exists in a used space.

There are 5 spaces; 1, 2, 3, 4, 5

At any point, one index on the spinning platter exists, in each clock digit space. Due to the spacing of the platter design, only one segment can exist in the clock space at a time. This is so we don't produce noise in the other clock spaces.

To know if we need to strobe a segment into a clock space, we need to know the digit currently displayed in each clock space

Digit1:
Digit2:
Digit3:
Digit4:
Digit5:

At each strobe point, we check what digit is in each clock space, which segment exists in the clock space, and if that segment is used in the clock. If it is, we strobe on that index, if not, no strobe

That means for each rotation, we pre-simulate the rotation. This is because the timer requires all the strobe data at once. It then syncs that data with the hall sensor. We need a function that will generate all 105 frames that we can send to the timer.

Start by assuming
Clock exists in spaces 0, 1, 2, 3, 4
Hall sensor triggers on platter index 0
105 slices
Digits are: 7, 7, 7, 7, 7

We need to define which platter indexes are required to make each digit (what segments are used)
Lets define each one as a byte structure. We wont use the MSB, always set it to 0.
digitDesign = 0b0ABCDEFG

Each binary digit space in the byte, represents each segment with the pattern being A, B, C, D, E, F, G

digitDesign_0 = 0b01111110
digitDesign_1 = 0b00110000
digitDesign_2 = 0b01111001

```
digitDesign_3 = 0b01111001
digitDesign_4 = 0b00110011
digitDesign_5 = 0b01011011
digitDesign_6 = 0b01011111
digitDesign_7 = 0b01110000
digitDesign_8 = 0b01111111
digitDesign_9 = 0b01111011
digitDesign_- = 0b00000001
```

For now, the simplest way seems to be to simulate the entire rotation, and check the conditions (based on current clock digit values) if the strobe would be on or off for each slice index. We then fill the buffer based on the simulation and end it out to the timer

Write some sudo code

```
For the length of 105 slices
    Digit1CurrentSlice = currentSlice + 0 //adding my offset here later will allow me to position the
    clock display
    Digit2CurrentSlice = currentSlice + 1
    Digit3CurrentSlice = currentSlice + 2
    Digit4CurrentSlice = currentSlice + 3
    Digit5CurrentSlice = currentSlice + 4

    (write functions that can check if a slice exists in a digit)
    Digit1Check = checkDigits(digit1, digit1CurrentSlice)
    Digit2Check = checkDigits(digit2, digit2CurrentSlice)
    Digit3Check = checkDigits(digit3, digit3CurrentSlice)
    Digit4Check = checkDigits(digit4, digit4CurrentSlice)
    Digit5Check = checkDigits(digit5, digit5CurrentSlice)

    //if a used segment exists in a clock space, turn on the strobe for that space
    If(digit1Check || digit2Check || digit3Check || digit4Check || digit5Check)
        //turn the strobe on for this frame
        frameBuffer[currentSlice, 1]
    Else
        frameBuffer[currentZlice, 0]
```

Running checklist

- ◯ Pull strobe mode from NFC
- ◯ Act on strobe mode value

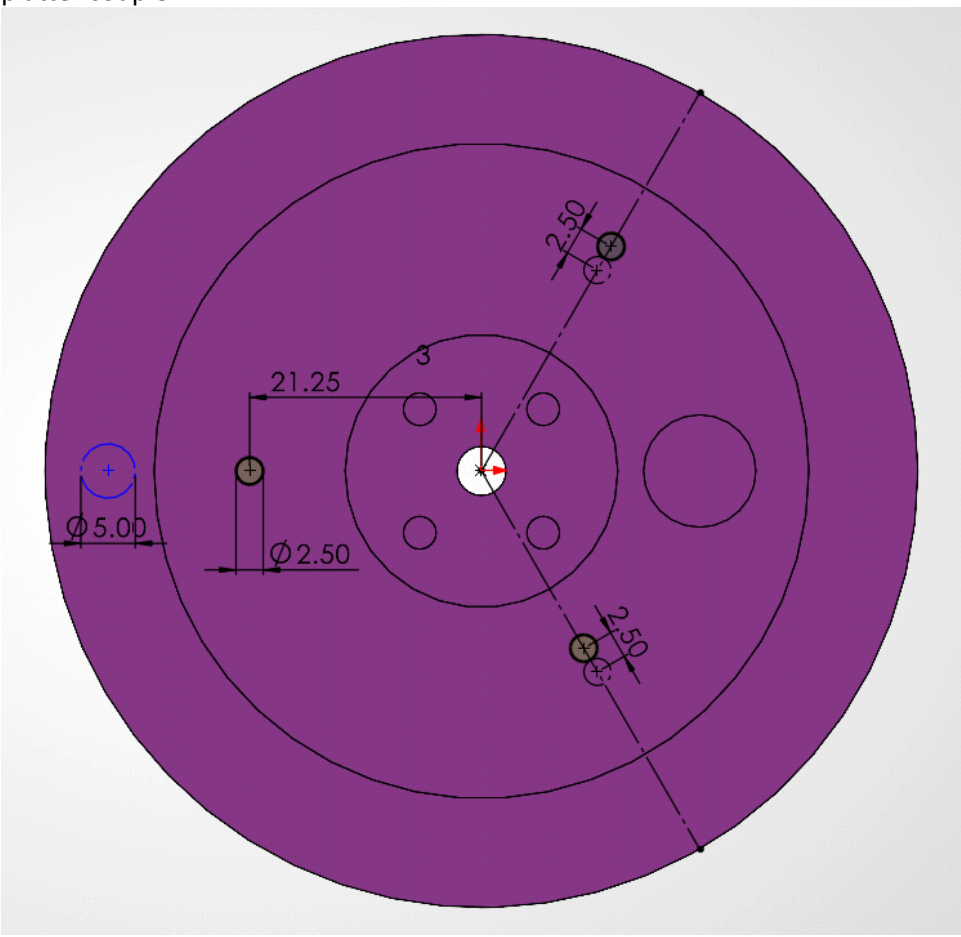
Platter Requirements

Wednesday, March 4, 2026 8:26 PM

Some platters are more complicated than others. Specific platters require strict indexing to work properly. This is done with spring pins in the platter coupler.

When designing a platter, include holes in the platter to index to these pins.

The index point in software is the magnet. That is the hole on the right side in the below image. The left spring pin is directly opposite from the magnet. The holes are spread evenly around the disk. The pins vary in distance from the center of the platter to allow for a single index point between the platter and platter coupler



Typical platters are 200mm in diameter. The largest of platters are as large as 135mm

LED Gooseneck Spotlight "Saber"

Thursday, March 12, 2026 7:09 PM

I have added an XLR 3 pin connector to the top of the zoetrope that passes the strobed ground signal, 12V power, and a third connection that activates a relay when plugged in. When plugged in the strobe signal automatically switches, via the relay, to the gooseneck LED strip. This allows for a spotlight effect allowing for a specific part of the spinning platter to be highlighted.

ADD IMAGE HERE

The original gooseneck light was not nearly bright enough. The effect came through, but it is increadibly dim. I will create a new end effector for the gooseneck using 5000k 95-CRI LEDs on a homemade CNC milled circuit board(s). I want to add a cool effect to the back of the effector (normally faces up towards the user). I will embed a short string of red cob lights in the back of the effector giving a "light saber" effect. It would also be worth trying the normal warm white for the back as well, it may fit the color scheme better

Good reference for LED strip light designs
https://www.waveformlighting.com/pcb-designs/led-strip-light-schematic-and-voltage-information

I am using 12V to supply the LEDs. We will use 3 LEDs and one resistor.

This project is a huge success! Here are the main takeaways

- LEDs Used: **L130-5095001400001**

- The standard 3 pin XLR
  - Very robust, but has its limits. We need metal connectors to get the required strength to match our gooseneck
  - 3 pins is barely enough, using metal connectors gives us a fourth (connector shell) connection to use.
  - Embed the female connector in the back, top corner piece. Pass cable up tube to connector

- Gooseneck, get one hollow, about 10mm diameter, threaded ends
  - End 1 - 3 pin XLR male connector (metal)
  - End 2 - four pin magnet rotation connector, tracks ends

- End effectors
  - Has four pin magnet rotation connector, spring pins end
  - Multiple ends
    - Light bar
    - Spot light

- The three wires allows one wire to activate a relay to re-direct the strobe signal, and two connections to power the strobe